

School of Computer Engineering
Computer Engineering Bachelor
IC8059 - Introduction to Computer Graphics

Homework

Image Magick

Andrés Canossa Castro
acanossa@estudiantec.cr
2022008322

Binjie Liang
bliang@estudiantec.cr
2023064642

William Jiménez Venegas
wijimenez@estudiantec.cr
2023396021

San José, Costa Rica
September 2026

1 ImageMagick Installation

ImageMagick is an open-source software suite for creating, converting, modifying, and analyzing raster and vector images. At the time this work was performed, the most recent stable release available from the official ImageMagick distribution was version `7.1.2-31` [1].

The source code was obtained from the official ImageMagick GitHub repository and compiled using the standard configuration and installation procedure described in the ImageMagick documentation [2, 3]. The main commands used for the installation were [4]:

- git clone –depth 1 –branch 7.1.2-31 `https://github.com/ImageMagick/ImageMagick.git` ImageMagick-7.1.2-31
- cd ImageMagick-7.1.2-31
- ./configure
- make
- sudo make install
- sudo ldconfig

After the installation, the installed version was verified using: magick -version

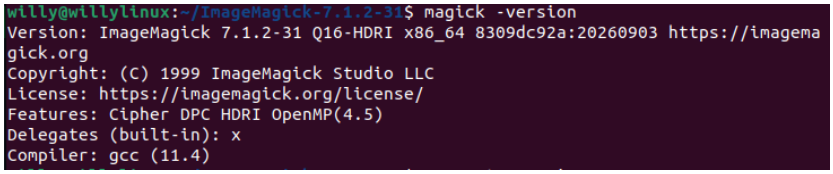


Figure 1: Verification of the installed ImageMagick version.

Several command-line operations were performed in order to become familiar with the main ImageMagick functionalities. First, an example image was generated using one of the images included with ImageMagick:

```
magick logo: original.png
```

Information about the image, such as its dimensions, format, color space, and bit depth, can be obtained with the `identify` command:

```
magick identify original.png
```

```
willy@willylinux:~/Downloads$ magick logo: original.png
willy@willylinux:~/Downloads$ ls
original.png
willy@willylinux:~/Downloads$ magick identify original.png
original.png PNG 640x480 640x480+0+0 8-bit sRGB 256c 27434B 0.000u 0:00.000
```

Figure 2: Basic commands 1.

ImageMagick can then apply a wide variety of transformations. Some of the operations tested were resizing, rotation, grayscale conversion, cropping, and blurring:

```
magick <input> -resize 50% <output>
magick <input> -rotate 20 <output>
magick <input> -colorspace Gray <output>
magick <input> -crop 300x200+20+20 <output>
magick <input> -blur 0x3 <output>
```

```
willy@willylinux:~/Downloads$ ls
original.png
willy@willylinux:~/Downloads$ magick original.png -resize 50% resized.png
willy@willylinux:~/Downloads$ magick original.png -rotate 20 rotated.png
willy@willylinux:~/Downloads$ magick original.png -colorspace Gray grayscale.png
willy@willylinux:~/Downloads$ magick original.png -crop 300x200+20+20 cropped.png
willy@willylinux:~/Downloads$ magick original.png -blur 0x3 blurred.png
willy@willylinux:~/Downloads$ ls
blurred.png  cropped.png  grayscale.png  original.png  resized.png  rotated.png
```

Figure 3: Basic commands 2.

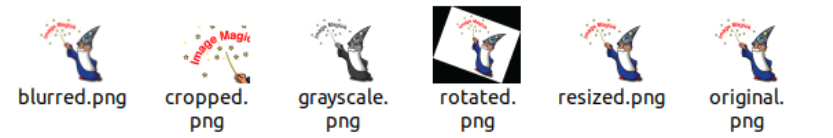


Figure 4: Example of the images created.

Figure 4 shows the results of these operations. These experiments illustrate that ImageMagick can be used not only for format conversion but also for image transformations, color-space manipulation, filtering, image composition, and image inspection.

The formats supported by the installed version can be inspected with:

```
magick identify -list format
```

In particular, the output contains the `AVS` format identified as *AVS X image*, with both reading and writing support [5].

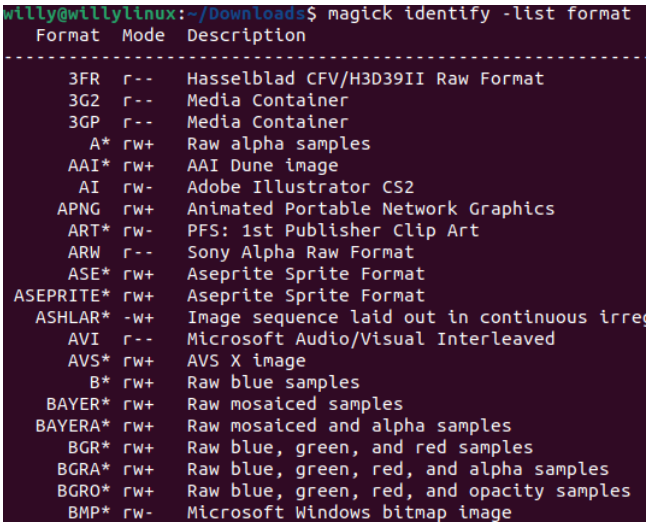


Figure 5: Example of some formats in Image Magick.

2 ImageMagick: AVS Format

ImageMagick identifies the `AVS` format as the *AVS X image* format and provides both reading and writing support for it [5]. The format should not be confused with video formats that use the same AVS acronym.

The AVS X-Image format originated on UNIX workstations and was commonly used to represent true-color raster images with transparency information. Each pixel contains four 8-bit components, giving a total of 32 bits, or four bytes, per pixel [6]. The four components are stored in the order alpha, red, green, and blue (A-R-G-B) [7, 8].

The internal structure of a single AVS X-Image is particularly simple. The file begins with an 8-byte header followed immediately by the raster pixel data [7]. The header contains two unsigned 32-bit integers representing the image width and height. Both integers are stored using most-significant-byte-first, or big-endian, byte ordering [8].

Pixels are stored from left to right and from the top of the image to the bottom [7]. For each pixel, one unsigned byte is stored for each of the four components: a == alpha, r == red, g = reen, b == blue, where each colour/alpha component is 1 unsigned byte [7].

Unlike formats such as PNG or JPEG, the basic AVS X-Image representation does not contain a complex metadata structure or compressed image stream. ImageMagick's AVS encoder writes the width (the first four bytes) and height (the next four bytes) directly, followed by the four channel bytes for every pixel [8].

For example, to verify the internal structure of the AVS format, the image show next will be converted from JPEG to AVS using ImageMagick.



Figure 6: Original image used for the AVS format experiment.

The image can be converted to .avs using:

```
magick smiley.jpg smiley.avs
```

```
willy@willylinux:~/Downloads$ magick smiley.jpg smiley.avs
magick: delegate library support not built-in `smiley.jpg' (XML) @ warning/profile.c/ValidateXMPProfile/2019.
willy@willylinux:~/Downloads$ ls
smiley.avs  smiley.jpg  Tarea_Corta_0  Tarea_Corta_0.zip
```

Figure 7: Image in .avs format.

The original image has a resolution of 612×612 pixels. The first bytes of the resulting AVS file can be examined with:

```
xxd -g 1 -l 32 smiley.avs
```

Since 612 in hexadecimal is `0x264`, its 32-bit big-endian representation is 00 00 02 64. Therefore, the first eight bytes of the AVS file are: 00 00 02 64 00 00 02 64.

```
willy@willylinux:~/Downloads$ xxd -g 1 -l 32 smiley.avs
00000000: 00 00 02 64 00 00 02 64 ff ff ff ff ff ff ff ff  ...d...d........
00000010: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff  ................
```

Figure 8: Bytes in a .avs format.

As a matter of fact, the term AVS is also associated with scientific-visualization formats that should not be confused with AVS X-Image. In particular, AVS Field files, commonly identified by the `.fld` extension, are designed to describe multidimensional scientific datasets rather than ordinary two-dimensional raster images.

Scientific-visualization research such as the work of Nonaka et al. discusses visualization of large-scale datasets on high-performance computing systems [9]. Such scientific data representations serve a different purpose from the AVS X-Image format implemented by ImageMagick.

3 Load Matrix with .avs

The function `load_avs` reads an AVS file and returns a matrix of pixels in memory.

As explained earlier, the AVS format stores the image width and height in the first eight bytes of the file, followed by the pixel data. The function reads this information and allocates memory for a matrix of pixels.

Given that we are working in a little-endian architecture, the function must convert the big-endian bytes of the header to little-endian integers.

First of all, we have to define a structure to represent a pixel in memory:

```
1  typedef struct {
2      unsigned char a; // alpha channel
3      unsigned char r; // red channel
4      unsigned char g; // green channel
5      unsigned char b; // blue channel
6  } Pixel;
```

This structure allows us to store the four channels of a pixel in a single object.

To start loading the image, we open the AVS file in binary read mode, then we read the first four bytes to obtain the image width, and the next four bytes to obtain the image height. We convert these bytes from big-endian to little-endian integers.

```
1      FILE *file = fopen(filename, "rb"); // read file as binaries
2
3      if (file == NULL) {
4          printf("No file found.\n");
5          return NULL;
6      }
7
8      // define a 4 bytes buffer to read the header
9      unsigned char buffer[4];
10
11     // read first 4 bytes of the header -> width
12     fread(buffer, 1, 4, file);
13
14     // arrange big endian bytes to little endian
15     width = ((int)buffer[0] << 24) |
16             ((int)buffer[1] << 16) |
17             ((int)buffer[2] << 8) |
18             buffer[3];
19
20     // read the follwing 4 bytes -> height
21     fread(buffer, 1, 4, file);
22
23     // arrange header big endian bytes to little endian
24     height = ((int)buffer[0] << 24) |
25             ((int)buffer[1] << 16) |
26             ((int)buffer[2] << 8) |
27             buffer[3];
```

After obtaining the dimensions, we allocate memory for a matrix of pixels with the specified width and height.

```
    Pixel **image = malloc(height *sizeof(Pixel *));

    for (int y=0; y < height; y++) {
        // for every y in height, create row
        image[y] = malloc(width * sizeof(Pixel));
    }
```

We then read the pixel data from the file, storing each pixel in the corresponding position in the matrix.

```
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            fread(buffer, 1, 4, file);
            image[y][x].a = buffer[0];
            image[y][x].r = buffer[1];
            image[y][x].g = buffer[2];
            image[y][x].b = buffer[3];
        }
    }
```

Finally, we close the file and return the pointer to the matrix of pixels.

4 Save .avs from memory

The function `save_avs` stores an image represented in memory as an AVS file. Its declaration is:

```
int save_avs(const char *filename, Pixel **image,
             int image_width, int image_height)
```

The function receives the output filename, a matrix of pixels, the image width, and the image height. It returns `0` if the operation is successful and `-1` if an error occurs.

4.1 Opening the output file

The file is opened in binary write mode:

```
FILE *file = fopen(filename, "wb");
```

The `w` flag enables writing, while the `b` flag indicates that the file must be treated as binary data. If the file cannot be created, the function reports the error and returns `-1`.

```
if (file == NULL) {
    printf("Could not create file.\n");
    return -1;
}
```

4.2 Writing the image width

The AVS format stores the width in the first four bytes of the file. These bytes must be written in big-endian order. The bit-shift operators extract each byte from the integer:

```
buffer[0] = (unsigned char)((image_width >> 24) & 0xff);
buffer[1] = (unsigned char)((image_width >> 16) & 0xff);
buffer[2] = (unsigned char)((image_width >> 8) & 0xff);
buffer[3] = (unsigned char)(image_width & 0xff);
```

The four bytes are then written to the file using `fwrite`:

```
if (fwrite(buffer, 1, 4, file) != 4) {
    fclose(file);
    return -1;
}
```

The return value of `fwrite` is checked to ensure that all four bytes were written successfully.

4.3 Writing the image height

The height is written using the same big-endian conversion:

```
buffer[0] = (unsigned char)((image_height >> 24) & 0xff);
buffer[1] = (unsigned char)((image_height >> 16) & 0xff);
buffer[2] = (unsigned char)((image_height >> 8) & 0xff);
buffer[3] = (unsigned char)(image_height & 0xff);
```

The resulting four bytes are written immediately after the width. The first eight bytes of the AVS file contain the image dimensions:

```
4 bytes for the width
4 bytes for the height
```

4.4 Writing the pixel data

After writing the header, the function iterates through every row and column of the image:

```
for (int y = 0; y < image_height; y++) {
    for (int x = 0; x < image_width; x++) {
```

Each pixel is stored in the order alpha, red, green, and blue:

```
buffer[0] = image[y][x].a;
buffer[1] = image[y][x].r;
buffer[2] = image[y][x].g;
buffer[3] = image[y][x].b;
```

Each pixel occupies four bytes in the output file. The function writes those bytes and verifies that the operation succeeds:

```
if (fwrite(buffer, 1, 4, file) != 4) {
    fclose(file);
    return -1;
}
```

Pixels are written from left to right and from the top row to the bottom row.

4.5 Closing the file

Once the header and all pixel data have been written, the file is closed:

```
if (fclose(file) != 0) {
    return -1;
}
```

If the file is closed successfully, the function returns zero:

```
return 0;
```

4.6 Using the function in `main`

The image is first loaded into memory using `load_avs`:

```
Pixel **image = load_avs("image.avs");
```

The in-memory image is then saved as a new AVS file:

```
if (save_avs("image_copy.avs", image, width, height) == 0) {
    printf("Image saved successfully.\n");
} else {
    printf("Could not save image.\n");
}
```

This creates `image_copy.avs` with the same dimensions and pixel values as the original image.

Finally, the memory allocated for the pixel matrix is released row by row:

```
for (int y = 0; y < height; y++) {
    free(image[y]);
}
free(image);
```

This prevents memory leaks after the image has been saved.

References

[1] ImageMagick Studio LLC, "Download imagemagick," ImageMagick, 2026, accessed: 2026-09-11. [Online]. Available: https://imagemagick.org/download/

[2] ——, "Install imagemagick from source," ImageMagick, 2026, accessed: 2026-09-11. [Online]. Available: https://imagemagick.org/install-source/

[3] ——, "Imagemagick source code repository," GitHub, 2026, accessed: 2026-09-11. [Online]. Available: https://github.com/ImageMagick/ImageMagick

[4] S. Sharma, "Install imagemagick on ubuntu," It's FOSS, May 2025, published: 2025-05-26. Accessed: 2026-09-11. [Online]. Available: https://itsfoss.com/install-imagemagick-ubuntu/

[5] ImageMagick Studio LLC, "Image formats," ImageMagick, 2026, accessed: 2026-09-11. [Online]. Available: https://imagemagick.org/formats/

[6] Golden Software, "Avs x-image [.ximg] file description," Grapher Help, 2026, accessed: 2026-09-11. [Online]. Available: https://grapherhelp.goldensoftware.com/subsys/subsys_avs_x-image_file_desc.htm

[7] P. Bourke, "Avs image format," Jan. 2001, accessed: 2026-09-11. [Online]. Available: https://paulbourke.net/dataformats/avs_x/

[8] ImageMagick Studio LLC, "Imagemagick avs x-image coder," GitHub, 2026, accessed: 2026-09-11. [Online]. Available: https://github.com/ImageMagick/ImageMagick/blob/main/coders/avs.c

[9] J. Nonaka, M. Matsuda, T. Shimizu, N. Sakamoto, M. Fujita, K. Onishi, E. C. Inacio, S. Ito, F. Shoji, and K. Ono, "A study on open source software for large-scale data visualization on sparc64fx based hpc systems," in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*. New York, NY, USA: Association for Computing Machinery, 2018, pp. 278–288.